

Entrega 2

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

Esta segunda entrega ya da el salto a las estructuras de control y a los formularios, que es donde PHP empieza a tener sentido de verdad. En la primera entrega aprendimos a "imprimir cosas", pero aquí ya hay decisiones (`if`, `switch`, `match`), bucles, funciones reutilizables, arrays más serios y formularios que reciben datos del usuario.

Los programas van del 11 al 34, y se pueden agrupar más o menos así:

- Decisiones (11, 12, 13): `if/else`, `switch`, `match`.
 - Bucles (14, 15): `for`, `while`, `do/while`, `foreach`.
 - Funciones (16, 17, 18, 19): declaración, argumentos, referencias, `include/require`.
 - Funciones útiles (20): cadenas, fechas, números y arrays.
 - Arrays (21, 22, 23, 24): asociativos, bidimensionales, punteros internos y CRUD.
 - Formularios (25, 26, 27, 28, 29): `$_POST`, `$_GET`, `$_REQUEST` y validación.
 - Inputs (30, 31, 32, 33): login, radios, checkboxes y select.
 - Validación de email (34).
-

Programa 11

Sentencias condicionales `if`, `else` y `elseif`. Hay dos ejemplos en el archivo.

El primer ejemplo crea un usuario y una edad, y comprueba si es mayor o menor de edad:

Programa11.php · líneas 9-21

```
9      <?php
10      $usuario = "Francisco";
11      $edad = 20;
12
13      // Mostramos el usuario
14      echo "<h2>Bienvenido, $usuario</h2>";
15
16      if ($edad >= 18) {
17          echo "$usuario es mayor de edad.";
18      } else {
19          echo "$usuario es menor de edad.";
20      }
21      ?>
```

El segundo ejemplo compara dos variables `a` y `b` con `if`, `else if` y `else`:

Programa11.php · líneas 25-36

```
25      <?php
26      $a = 5;
27      $b = 10;
28
29      if ($a > $b) {
30          echo "a es más grande que b";
31      } else if ($a < $b) {
32          echo "b es más grande que a";
33      } else {
34          echo "a y b son iguales";
35      }
36      ?>
```

Salida:

Bienvenido, Francisco

Francisco es mayor de edad.

b es más grande que a

Programa 12

Sentencia **switch** con un caso muy básico: según el valor de **\$dia**, imprime un día de la semana. El detalle importante es el **break**: si se te olvida, los casos siguientes también se ejecutan.

```
1 <?php
2 $dia = 3;  // Cambia este valor para probar
3
4 switch ($dia) {
5
6     case 1:
7         echo "Lunes";
8         break;
9     case 2:
10        echo "Martes";
11        break;
12    case 3:
13        echo "Miércoles";
14        break;
15    default:
16        echo "Otro día";
17 }
18
```

Salida:

Miércoles

Programa 13

Expresión `match`, que es la versión moderna del `switch`. La diferencia es que `match` devuelve un valor que se puede asignar a una variable, es estricto en tipos (`===`) y no hace falta `break`.

Primer ejemplo, con un videojuego que se clasifica según su género:

Programa13.php · líneas 10-22

```
10  <?php
11  $videojuego = "Zelda";
12
13  $genero = match ($videojuego) {
14      "FIFA" => "Deportes",
15      "Call of Duty" => "Shooter",
16      "Zelda" => "Aventura",
17      "Minecraft" => "Sandbox",
18      default => "Género desconocido",
19  };
20
21  echo "El juego $videojuego pertenece al género: $genero";
22  ?>
```

Segundo ejemplo, con un usuario y un rol, donde el mensaje cambia según el rol:

Programa13.php · líneas 25-37

```
25  <?php
26  $usuario = "Carlos";
27  $rol = "admin";
28
29  $mensaje = match ($rol) {
30      "admin" => "Hola $usuario, tienes acceso completo al sistema.",
31      "editor" => "Hola $usuario, puedes editar contenido.",
32      "invitado" => "Hola $usuario, solo puedes ver la información.",
33      default => "Rol desconocido para $usuario.",
34  };
35
36  echo $mensaje;
37  ?>
```

Tercer ejemplo, donde el resultado del `match` se guarda en `$return_value`:

```
39     <?php
40     $food = 'cake';
41
42     $return_value = match ($food) {
43         'apple' => 'This food is an apple',
44         'bar' => 'This food is a bar',
45         'cake' => 'This food is a cake',
46     };
47
48     var_dump($return_value);
49     ?>
```

Salida:

El juego Zelda pertenece al género: Aventura

Hola Carlos, tienes acceso completo al sistema.
string(19) "This food is a cake"

Programa 14

Los tres bucles principales: `for`, `while` y `do...while`. El truco para saber cuál usar:

- `for` cuando sabes cuántas veces vas a iterar.
- `while` cuando iteras mientras se cumpla una condición.

- **do...while** cuando quieres que se ejecute al menos una vez, aunque la condición sea falsa desde el principio.

Ejemplo con **for**:

Programa14.php · líneas 1-7

```
1 <?php
2 // Programa14.php
3 echo "<h3>Ejemplo con for</h3>";
4 for ($i = 1; $i <= 5; $i++) {
5     echo "Número: $i <br>";
6 }
7
```

Ejemplo con **while**:

Programa14.php · líneas 8-13

```
8 echo "<h3>Ejemplo con while</h3>";
9 $j = 2;
10 while ($j <= 10) {
11     echo "Par: $j <br>";
12     $j += 2;
13 }
```

Ejemplo con **do...while**:

Programa14.php · líneas 15-21

```
15 echo "<h3>Ejemplo con do...while</h3>";
16 $k = 5;
17 do{
18     echo "Cuenta atrás: $k <br>";
19     $k--;
20 } while ($k >= 0);
21 ?>
```

Salida:

Ejemplo con for

Número: 1
Número: 2
Número: 3
Número: 4
Número: 5

Ejemplo con while

Par: 2
Par: 4
Par: 6
Par: 8
Par: 10

Ejemplo con do...while

Cuenta atrás: 5
Cuenta atrás: 4
Cuenta atrás: 3
Cuenta atrás: 2
Cuenta atrás: 1
Cuenta atrás: 0

Programa 15

`foreach` aplicado a varios casos. Lo bueno de `foreach` es que no tienes que llevar tú el índice, y si pones `as $clave => $valor` te da las dos cosas a la vez.

Recorrido simple y formato clave => valor:

Programa15.php · líneas 1-19

```
1 <?php
2 // Array simple de videojuegos
3 $videojuegos = ["Zelda", "FIFA", "Minecraft", "Call of Duty"];
4
5 // --- Foreach en su forma simple (solo valores)
6 echo "<h3>Lo mostramos con print_r()</h3>";
7 print_r($videojuegos);
8 echo "<br>";
9
10 echo "<h3>Recorrido simple con foreach</h3><br>";
11 foreach ($videojuegos as $juego) {
12     echo "Juego: $juego <br>";
13 }
14
15 // --- Foreach con clave => valor
16 echo "<h3>Recorrido con FOREACH clave => valor</h3>";
17 foreach ($videojuegos as $indice => $juego) {
18     echo "Índice $indice => $juego <br>";
19 }
```

Matriz bidimensional (un array de arrays asociativos con **nombre** y **genero**):

Programa15.php · líneas 21-37

```
21 // --- Matriz bidimensional: videojuegos con género
22 $listaJuegos = [
23     ["nombre" => "Zelda", "genero" => "Aventura"],
24     ["nombre" => "FIFA", "genero" => "Deportes"],
25     ["nombre" => "Minecraft", "genero" => "Sandbox"],
26     ["nombre" => "Call of Duty", "genero" => "Shooter"]
27 ];
28
29 echo "<h2>Matriz bidimensional</h2>";
30 echo "<h3>Lo mostramos con print_r()</h3>";
31 print_r($listaJuegos);
32 echo "<br>";
33 echo "<h3>Recorrido de matriz bidimensional</h3>";
34 foreach ($listaJuegos as $item) {
35     echo "El juego {$item['nombre']} es de género {$item['genero']} <br>";
36 }
37
```

Último bloque, recorriendo **\$_SERVER** con **foreach**:

```
Programa15.php · líneas 40-46

40 <?php
41 echo "<h3>Mostrando variables del servidor</h3>";
42
43 foreach ($_SERVER as $clave => $valor) {
44     echo "<strong>CLAVE:</strong> $clave => <strong>VALOR:</strong> $valor <br>";
45 }
46 ?>
```

Salida:

Lo mostramos con print_r()

```
Array ( [0] => Zelda [1] => FFA [2] => Masecraft [3] => Call of Duty )
```

Recorrido simple con foreach

Recorrido simple con foreach

```

Juego Zelda
Juego TFA
Juego Minecraft
Juego Call of Duty

```

```
Recorrido con FOREACH clave => valor
Índice 0 => Zaida
Índice 1 => FFFA
Índice 2 => Manuash
Índice 3 => Call of Duty
```

Matriz bidimensional

Lo mostramos con `print_r()`

```
Array ( [0] => Array ( [nombre] => Zaida [genero] => Aventuras ) [1] => Array ( [nombre] => FIFA [genero] => Deportes ) [2] => Array ( [nombre] => Mascarita [genero] => Sanfuchs ) [3] => Array ( [nombre] => Call of Duty [genero] => Shooter ) )
```

```

La mostramos con print_f()

Array ( [0] => Array ( [nombre] => Zelda [genero] => Aventura ) [1] => Array ( [nombre] => FIFA [genero] => Deportes ) [2] => Array ( [nombre] => Mascaral [genero] => Sandbox ) [3] =>
Array ( [nombre] => Call of Duty [genero] => Shooter ) )

Recorrido de matriz bidimensional

```

Mostrando variables del servidor

```
C:\AUG>SET /V ?>C:\AUG\B1\comandos\info\server\info.txt
```

[illegible]

Programa 16

Cómo declarar funciones. Hay dos casos en el archivo.

Función simple sin argumentos, que calcula el precio con IVA usando valores fijos

Programa16.php · líneas 1-11

```
1 <?php
2 // Función simple sin argumentos
3 function precioConIVA(): float {
4     $precio= 100; // definido dentro de la función
5     $iva = 21;     // definido dentro de la función
6     $precioFinal = $precio + ($precio * $iva / 100);
7     return round($precioFinal, 2);
8 }
9
10 // Ejemplo de uso
11 echo "Precio con IVA: " . precioConIVA() . " €<br>";
```

Función condicional: solo se define si se cumple un **if**. Por dentro usa **global** para acceder a una variable externa:

Programa16.php · líneas 14-28

```
14 $iva = true;
15 $precio = 10;
16 print "Llamada antes de la condicional";
17
18 if ($iva) {
19     function precio_con_iva_condi(){
20         global $precio;
21         $precioiva = $precio * 1.21;
22         print "El precio con IVA es $precioiva";
23     }
24 }
25
26 print "Llamada después de la condicional";
27 precio_con_iva_condi();
28 ?>
```

Salida:

Precio con IVA: 121 €
Llamada antes de la condicionalLlamada después de la condicionalEl precio con IVA es 12.1

Programa 17

Misma función `precioConIVA` pero ya con argumentos tipados (`float $precio`, `float $iva`). El tipado en PHP es opcional pero ayuda a leer el código y a evitar errores tontos.

Programa17.php · líneas 1-14

```
1 <?php
2 function precioConIVA (float $precio , float $iva ): float {
3     // $precio → precio base
4     // $iva     → porcentaje de IVA (por defecto 21%)
5     $precioFinal = $precio + ($precio * $iva / 100);
6     return round($precioFinal, 2); // Redondeamos a 2 decimales
7 }
8
9 // Ejemplo de uso
10 $precioBase = 100;
11 echo "Precio base: $precioBase €<br>";
12 echo "Precio con IVA (21%): " . precioConIVA($precioBase, 21) . " €<br>";
13 echo "Precio con IVA (10%): " . precioConIVA($precioBase, 10) . " €<br>";
14 ?>
```

Salida:

Precio base: 100 €
Precio con IVA (21%): 121 €
Precio con IVA (10%): 110 €

Programa 18

Argumentos por referencia con `&`. La diferencia con el paso por valor es que aquí no se pasa una copia, sino la dirección de memoria, así que si la función cambia la variable, también cambia fuera.

```

1 <?php
2     print "<br/><br/> ARGUMENTOS POR valor y por referencia <br /> ";//ARGUMENTOS POR DEFECTO
3     function precio_iva_referencia (&$precio /*Le pasas su direccion de memoria 100325*/, $iva=0.21) {
4         $precio *= (1 + $iva);
5         RETURN $precio;
6     }
7
8     $precio = 10; //imagina que su MEMORY ADDRESS vale 100325
9     print "<br/><br/>1- ANTES de llamar a la función: El precio con IVA es ".$precio ; //10
10
11     precio_iva_referencia($precio);
12
13     print "<br/>2- DESPUES de llamar a la función: El precio con IVA es ". $precio ; //12.1
14     print "<br/><b>3- Anota en tus apuntes qué RETURN has usado</b>"
15
16 ?>

```

Salida:

ARGUMENTOS POR valor y por referencia

1- ANTES de llamar a la función: El precio con IVA es 10
 2- DESPUES de llamar a la función: El precio con IVA es 12.1
 3- **Anota en tus apuntes qué RETURN has usado**

Programa 19

`include`, `require`, `include_once` y `require_once`. Sirven para reutilizar código de otros archivos.

Comando	Si el fichero falla	Se puede incluir varias veces
<code>include</code>	Solo avisa, sigue.	Sí.
<code>require</code>	Fatal error, se para.	Sí.
<code>include_once</code>	Solo avisa.	No, solo una vez.
<code>require_once</code>	Fatal error.	No, solo una vez.

Primero se declara `funciones.php`, que tiene `saludar`, `despedir`, `multiplicar` y `dividir`:

`funciones.php` · líneas 1-20

```
1 <?php
2 function saludar($nombre) {
3     return "Hola, $nombre. Bienvenido.";
4 }
5
6 function despedir($nombre) {
7     return "Adiós, $nombre. Hasta pronto.";
8 }
9
10 function multiplicar($a, $b) {
11     return $a * $b;
12 }
13
14 function dividir($a, $b) {
15     if ($b == 0) {
16         return "Error: división por cero";
17     }
18     return $a / $b;
19 }
20 ?>
```

Programa 19A, que usa `include`:

`Programa19A.php` · líneas 1-8

```
1 <?php
2 echo "<h2>Pruebas de include / require</h2>";
3
4 // 1. include
5 include "funciones.php";
6 echo saludar("Francisco") . "<br>";
7
8 ?>
```

Programa 19B, que usa `require`:

Programa19B.php · líneas 1-9

```
1 <?php
2 echo "<h2>Pruebas de include / require</h2>";
3
4 // 2. require
5 require "funciones.php";
6 echo despedir("Luis") . "<br>";
7
8
9 ?>
```

Programa 19C, que usa `include_once` dos veces (la segunda se ignora):

Programa19C.php · líneas 1-7

```
1 <?php
2 echo "<h2>Pruebas de include / require</h2>";
3 // 3. include_once (solo se incluirá una vez)
4 include_once "funciones.php";
5 include_once "funciones.php"; // se ignora
6 echo "3 x 4 = " . multiplicar(3, 4) . "<br>";
7 ?>
```

Resultados en el navegador:

Pruebas de include / require

Hola, Francisco. Bienvenido.

Pruebas de include / require

Adiós, Luis. Hasta pronto.

Programa 20

Recopilación de funciones útiles de PHP que es importante conocer:

- `strlen()` para la longitud de una cadena.
- `strtoupper()` y `strtolower()` para mayúsculas y minúsculas.
- `substr()` para extraer parte de una cadena.
- `date()` para la fecha y hora.
- `number_format()` para formatear números con separadores.
- `rand()` para un número aleatorio en un rango.
- `array_sum()` para sumar valores de un array.
- `in_array()` para comprobar si un valor está en un array.
- `implode()` para convertir un array en una cadena.

Primer bloque (cadenas, fechas y números):

```
1 <?php
2 echo "<h2>Programa20 - Funciones más usadas en PHP</h2><br>";
3
4 // 1. strlen() → Longitud de una cadena
5 $cadena = "Programación en PHP";
6 echo "La longitud de '$cadena' es: " . strlen($cadena) . "<br>";
7
8 echo "<hr>";
9
10 // 2. strtoupper() → Convertir a mayúsculas
11 echo "En mayúsculas: " . strtoupper($cadena) . "<br>";
12
13 echo "<hr>";
14
15 // 3. strtolower() → Convertir a minúsculas
16 echo "En minúsculas: " . strtolower($cadena) . "<br>";
17
18 echo "<hr>";
19
20 // 4. substr() → Extraer parte de una cadena
21 echo "Subcadena (0 a 11): " . substr($cadena, 0, 11) . "<br>";
22
23 echo "<hr>";
24
25 // 5. date() → Fecha actual en formato personalizado
26 echo "Fecha actual: " . date("d/m/Y H:i:s") . "<br>";
27
28 echo "<hr>";
29
30 // 6. number_format() → Formatear un número con separadores
```

Segundo bloque (aleatorios y arrays):

```

31 $precio = 12345.6789;
32 echo "Precio formateado: " . number_format($precio, 2, ",", ".") . " €<br>";
33
34 echo "<hr>";
35
36 // 7. rand() → Número aleatorio entre un rango
37 echo "Número aleatorio entre 1 y 100: " . rand(1, 100) . "<br>";
38
39 echo "<hr>";
40
41 // 8. array_sum() → Sumar todos los valores de un array
42 $numeros = [2, 4, 6, 8, 10];
43 echo "Suma de [2,4,6,8,10]: " . array_sum($numeros) . "<br>";
44
45 echo "<hr>";
46
47 // 9. in_array() → Verificar si un valor está en un array
48 $colores = ["rojo", "verde", "azul"];
49 echo "¿Existe 'verde' en el array de colores? " . (in_array("verde", $colores) ? "Sí" : "No") . "<br>";
50
51 echo "<hr>";
52
53 // 10. implode() → Convertir array a cadena
54 echo "Colores en un array: " . implode(", ", $colores) . "<br>";

```

Salida:

Programa20 - Funciones más usadas en PHP

La longitud de 'Programación en PHP' es: 20

En mayúsculas: PROGRAMACIÓN EN PHP

En minúsculas: programación en php

Subcadena (0 a 11): Programaci💎

Fecha actual: 09/06/2026 16:37:03

Precio formateado: 12.345,68 €

Número aleatorio entre 1 y 100: 33

Suma de [2,4,6,8,10]: 30

¿Existe 'verde' en el array de colores? Sí

Colores en un array: rojo, verde, azul

Programa 21

Arrays asociativos. Se ve cómo se pueden declarar de dos formas, con `array(...)` y con `[...]`, y también cómo construir un array paso a paso (con índice explícito o sin él).

Definición de `$array1` y `$array2`:

Programa21.php · líneas 1-28

```
1 <?php
2 // Ejemplo de definición de array simple
3
4 $array1 = array(
5     "foo" => "bar",
6     "bar" => "foo",
7 );
8
9 // a partir de PHP 5.4
10 $array2 = [
11     "foo" => "bar",
12     "bar" => "foo",
13 ];
14
15 // Mostrar todo el array 1
16 echo "<pre>";
17 print_r($array1);
18 echo "</pre>";
19
20 // Mostrar un valor específico
21 echo "Valor de foo en array1: " . $array1["foo"] . "<br>";
22 echo "Valor de bar en array1: " . $array1["bar"] . "<br>";
23
24 // Mostrar todo el array 2
25 echo "<pre>";
26 print_r($array2);
27 echo "</pre>";
28
```

Arrays paso a paso (`$modulos1` con índices, `$modulos2` sin índice):

```
29 //Array creado paso a paso
30 $modulos1[0] = "Programación";
31 $modulos1[1] = "Bases de datos";
32 $modulos1[2] = "Desarrollo web en entorno servidor";
33
34 //Array creado paso a paso sin decir posición
35 $modulos2[] = "Programación";
36 $modulos2[] = "Bases de datos";
37 $modulos2[] = "Desarrollo web en entorno servidor";
38
39 echo " <br> Modulos 2: ";
40 print_r($modulos2);
41
42 echo " <br> ";
43
44 ?>
```

Salida:

```
Array
(
    [foo] => bar
    [bar] => foo
)
Valor de foo en array1: bar
Valor de bar en array1: foo
Array
(
    [foo] => bar
    [bar] => foo
)

Modulos 2: Array ( [0] => Programación [1] => Bases de datos [2] => Desarrollo web en entorno servidor )
```

Programa 22

Recorrer arrays. Aquí hay un array numérico (`$modulos1`) y uno asociativo bidimensional (`$ciclos`, con DAW y DAM dentro). Para el numérico se usa un `for`, y para el bidimensional un `foreach` anidado.

Array numérico con `for`:

Programa22.php · líneas 1-16

```
1 <?php
2 $ciclos = array(
3     "DAW" => array( "PR" => "Programación", "BD" => "Bases de datos", "DWES" => "Desarrollo web en entorno servidor"),
4     "DAM" => array( "PR" => "Programación", "BD" => "Bases de datos", "SGE" => "Sistemas de gestión empresarial")
5 );
6
7 //Array creado paso a paso
8 $modulos1[0] = "Programación";
9 $modulos1[1] = "Bases de datos";
10 $modulos1[2] = "Desarrollo web en entorno servidor";
11
12 //Mostrar elementos de array numérico
13 echo "<h3>Array numérico</h3>";
14 for ($i = 0; $i < count($modulos1); $i++) {
15     echo "Módulo $i: " . $modulos1[$i] . "<br>";
16 }
```

Array asociativo bidimensional con `foreach` anidado:

Programa22.php · líneas 18-32

```
18 //Mostrar elementos de array asociativo
19 echo "<h3>Array asociativo</h3>";
20 foreach ($ciclos ['DAW'] as $clave => $valor) {
21     echo "Clave $clave => $valor <br>";
22 }
23
24 echo "<h3>Array asociativo</h3>";
25 foreach ($ciclos as $ciclos => $modulos) {
26     echo "Ciclo $ciclos<br>";
27     foreach ($modulos as $clave => $valor) {
28         echo "    Módulo $clave => $valor <br>";
29     }
30     echo "<br>";
31 }
32
```

Salida:

Array numérico

Módulo 0: Programación
Módulo 1: Bases de datos
Módulo 2: Desarrollo web en entorno servidor

Array asociativo

Clave PR => Programación
Clave BD => Bases de datos
Clave DWES => Desarrollo web en entorno servidor

Array asociativo

Ciclo DAW
Módulo PR => Programación
Módulo BD => Bases de datos
Módulo DWES => Desarrollo web en entorno servidor

Ciclo DAM
Módulo PR => Programación
Módulo BD => Bases de datos
Módulo SGE => Sistemas de gestión empresarial

Programa 23

Funciones para mover el puntero interno de un array: `reset()`, `next()`, `prev()`, `end()` y `key()`. Cada array tiene un cursor invisible, y estas funciones lo mueven.

Movimiento normal del puntero (al primero, siguiente, anterior y último):


```

1 <?php
2 // Definir el array con ciudades
3 $ciudades = array("Sevilla", "Granada", "Córdoba", "Málaga", "Cádiz");
4
5 /* Programa23: COPIA COLOCA LAS FUNCIONES DONDE CORRESPONDADN :
6 next($ciudades)
7 reset($ciudades)
8 prev($ciudades)
9 end($ciudades)
10 */
11
12 // 1. "FUNCION ARRAY ????" → primer elemento
13 echo "Primer elemento con : " . reset($ciudades) . "<br>";
14 echo "Clave actual: " . key($ciudades) . "<br><br>";
15
16 // 2. "FUNCION ARRAY ????" → avanzar
17 echo "Siguiente Elemento con : " . next($ciudades) . "<br>";
18 echo "Clave actual: " . key($ciudades) . "<br><br>";
19
20 // 3. "FUNCION ARRAY ????" → retroceder
21 echo "Elemento con : " . prev($ciudades) . "<br>";
22 echo "Clave actual: " . key($ciudades) . "<br><br>";
23
24 // 4. "FUNCION ARRAY ????" → último elemento
25 echo "Último elemento con : " . end($ciudades) . "<br>";
26 echo "Clave actual: " . key($ciudades) . "<br><br>";

```

Qué pasa cuando se llama a `next()` después del último elemento (la clave actual pasa a ser `null`):

```

28 // 5. next() después del último → fuera del array
29 $valor = next($ciudades);
30 if ($valor === false && key($ciudades) === null) {
31     echo "El puntero está fuera del array (hemos pasado el final).<br>";
32 } else {
33     echo "Elemento actual: $valor<br>";
34 }
35
36 //Vamos a forzar a forzar que se muestre
37 echo "Elemento con next() fuera del array: " . next($ciudades) . "<br>";
38 echo "Clave actual: " . key($ciudades) . "<br><br>";

```

Salida:

Primer elemento con : Sevilla
Clave actual: 0

Siguiente Elemento con : Granada
Clave actual: 1

Elemento con : Sevilla
Clave actual: 0

Último elemento con : Cádiz
Clave actual: 4

El puntero está fuera del array (hemos pasado el final).
Elemento con next() fuera del array:
Clave actual:

Programa 24

Mini-CRUD sobre un array. El array es un concesionario de coches y hay que:

- Añadir un coche nuevo.
- Modificar uno existente.
- Eliminar otro con `unset()`.
- Reindexar si era un array numérico (con `array_values()`).
- Comprobar si una variable es array (`is_array()`), contar (`count()`), buscar valores (`in_array()`, `array_search()`) y claves (`array_key_exists()`).
- Mostrar el array final con `print_r()`.

Primer bloque (añadir, modificar, eliminar y reindexar):

```
1 <?php
2 // Array inicial con algunos coches
3 $concesionario = array(
4     "c1" => "Ford Fiesta",
5     "c2" => "Seat Ibiza",
6     "c3" => "Renault Clio"
7 );
8
9 // 1. Añadir un coche nuevo
10 $concesionario["c4"] = "Peugeot 208";
11
12 // 2. Modificar un coche existente
13 $concesionario["c2"] = "Seat León";
14
15 // 3. Eliminar un coche
16 unset($concesionario["c1"]); // Eliminamos el Ford Fiesta
17
18 // 4. Reindexar si fuera numérico
19 // (aquí no hace falta porque usamos claves de texto)
20 // Ejemplo con numérico:
21 $cochesNum = array("BMW", "Audi", "Mercedes");
22 unset($cochesNum[1]); // elimina "Audi"
23 $cochesNum = array_values($cochesNum); // reindexa desde 0
24
25 // 5. Comprobar si es array
26 if (is_array($concesionario)) {
27     echo "✓ Es un array<br>";
28 }else{
29     echo "X No es un array<br>";
30 }
```

Segundo bloque (búsquedas y claves):

```
32 // 6. Contar coches
33 echo "El concesionario tiene " . count($concesionario) . " coches<br>";
34
35 // 7. Buscar un coche concreto
36 if (in_array("Renault Clio", $concesionario)) {
37     echo "✓ El Renault Clio está en el concesionario<br>";
38 }
39
40 // 8. Buscar coche y obtener clave
41 $clave = array_search("Peugeot 208", $concesionario);
42 if ($clave !== false) {
43     echo "El Peugeot 208 tiene la clave: $clave<br>";
44 }
45
46 // 9. Comprobar si existe una clave
47 if (array_key_exists("c3", $concesionario)) {
48     echo "✓ Existe la clave 'c3' en el concesionario<br>";
49 }
50
51 // Mostrar el array final
52 echo "<pre>";
53 print_r($concesionario);
54 echo "</pre>";
55 ?>
```

Salida:

```
✓ Es un array
El concesionario tiene 3 coches
✓ El Renault Clio está en el concesionario
El Peugeot 208 tiene la clave: c4
✓ Existe la clave 'c3' en el concesionario
```

```
Array
(
    [c2] => Seat León
    [c3] => Renault Clio
    [c4] => Peugeot 208
)
```

Programa 25

Primer formulario con POST. El formulario tiene tres campos: nombre, email y mensaje. Cuando se envía, va a `accion.php` que comprueba que la petición sea POST y muestra los datos.

Detalles importantes que apunté:

- Hay que comprobar el método con `$_SERVER["REQUEST_METHOD"] === "POST"`.
- Conviene usar `htmlspecialchars()` para evitar que se inyecte HTML.
- Para los textos con saltos de línea, `n12br()` los convierte en `<br>`.

Código del formulario `form.html`:

 `code_prog25_form`

Código del procesador `accion.php`:

```
1 <?php
2 // form.php - recoge y muestra datos del formulario
3
4 //almacenamos las variables superglobales en variables locales tras tratar los caracteres especiales
5 // ?? '' indica que si no se ha recibido, se pone una cadena vacía
6 if ($_SERVER["REQUEST_METHOD"] === "POST") {
7     $nombre = htmlspecialchars($_POST['nombre'] ?? '');
8     $email = htmlspecialchars($_POST['email'] ?? '');
9     $mensaje = htmlspecialchars($_POST['mensaje'] ?? '');
10 }
11 ?>
12 <!doctype html>
13 <html lang="es">
14 <head>
15     <meta charset="utf-8">
16     <title>Datos recibidos</title>
17 </head>
18 <body>
19     <h1>Datos recibidos del formulario</h1>
20     <?php if (!empty($nombre) && !empty($email) && !empty($mensaje)): ?>
21         <p><strong>Nombre:</strong> <?php echo $nombre; ?></p>
22         <p><strong>Email:</strong> <?php echo $email; ?></p>
23         <p><strong>Mensaje:</strong><br><?php echo nl2br($mensaje); ?></p>
24     <?php else: ?>
25         <p style="color:red;">No se recibieron todos los datos correctamente.</p>
26     <?php endif; ?>
27 </body>
28 </html>
```

Formulario en el navegador:

Formulario de Francisco Pérez

Nombre:

Email:

Mensaje:

Programa 26

Formulario con checkboxes. Lo nuevo es que si en el `name` del input se pone `name="modulos[ ]"`, PHP lo recibe como un array. En el procesador se recoge con `$_POST[ 'modulos' ]` y se recorre con `foreach`.

Código del formulario:

code_prog26_form

Código del procesador con `$_POST`:

procesa.php · líneas 12-22

```
12 <?php
13     //Recogemos los valores de las variables.
14     $nombre = $_POST['nombre'];
15     $modulos = $_POST['modulos']; // esta es un array de valores
16
17     print "Nombre: ".$nombre."<br />";
18
19     foreach ($modulos as $clave => $modulo) {
20         print "Clave:". $clave . " Modulo: ".$modulo."<br />";
21     }
22 ?>
```

Formulario en el navegador:

Nombre del alumno:

Ciclos que cursa:

- ☐ Desarrollo web en entorno servidor
- ☐ Desarrollo web en entorno cliente

Programa 27

Demuestra qué hace `$_REQUEST`. Esta superglobal recoge datos de GET, POST y COOKIE a la vez. Es cómoda pero no se recomienda usarla en producción porque

mezcla orígenes y es peor para la seguridad. Lo correcto es usar `$_GET` o `$_POST` según el método del formulario.

Aquí el formulario manda los datos por GET, pero el procesador usa `$_POST` para el nombre (a propósito da error) y `$_REQUEST` para los módulos (sí funciona).

Código del formulario con `method="GET"`:

form.html · líneas 13-33

```
13 <body>
14     <!-- Formulario de nombre input, action es la que me relaciona con el bloque para procesar -->
15     <!-- method= post o get -->
16     <form name="input" action="procesa.php" method="GET">
17
18         <!--creará una variable nombre -->
19         Nombre del alumno: <input type="text" name="nombre" /><br />
20         <p>Ciclos que cursa:</p>
21
22         <!--creará un ARRAY modulos[] --> <!--DWES es el valor del 1er elemento del array y DWEC el segundo -->
23         <input type="checkbox" name="modulos[]" value="servidor" /> Desarrollo web en entorno servidor <br />
24         <input type="checkbox" name="modulos[]" value="cliente" /> Desarrollo web en entorno cliente<br />
25
26
27         <br />
28         <!--creará una variable con valor "Enviar" -->
29         <input type="submit" value="Enviar" />
30
31
32     </form>
33 </body>
```

Código del procesador, mezclando `$_POST` y `$_REQUEST`:

procesa.php · líneas 12-22

```
12 <?php
13     //Recogemos los valores de las variables.
14     $nombre = $_POST['nombre'];
15     $modulos = $_REQUEST['modulos']; // esta es un array de valores
16
17     print "Nombre: ".$nombre."<br />";
18
19     foreach ($modulos as $clave => $modulo) {
20         print "Clave:". $clave . " Modulo: ".$modulo."<br />";
21     }
22 ?>
```

Formulario en el navegador:

Nombre del alumno:

Ciclos que cursa:

- ☐ Desarrollo web en entorno servidor
- ☐ Desarrollo web en entorno cliente

Programa 28

Cómo mantener los valores del formulario después de enviarlo. Aquí se usan varios trucos:

- `empty()` para comprobar si un campo está vacío.
- `value="<?= ... ?>"` para meter en el `value` del input el dato que ya envió el usuario.
- `checked` en los checkboxes según si el valor está dentro del array enviado (`in_array(...)`).

Procesado POST y comprobación de aficiones:

```

1  <?php
2  // Procesar el formulario al enviarlo
3  if ($_SERVER["REQUEST_METHOD"] == "POST") {
4
5      // 1 Comprobar si el nombre está vacío
6      if (empty($_POST["nombre"])) {
7          echo "<p style='color:red;'>El campo nombre está vacío.</p>";
8      } else {
9          echo "<p>Nombre: " . htmlspecialchars($_POST["nombre"]) . "</p>";
10     }
11
12     // 2 Comprobar las aficiones (checkboxes)
13     if (!empty($_POST["aficiones"])) {
14         $aficiones = $_POST["aficiones"];
15         echo "<p>Aficiones seleccionadas: " . implode(", ", $aficiones) . "</p>";
16
17         // 3 Verificar si "deporte" está entre las seleccionadas
18         if (in_array("deporte", $aficiones)) {
19             echo "<p>Te gusta el deporte! </p>";
20         }
21     } else {
22         echo "<p>No has marcado ninguna afición.</p>";
23     }
24 }
25 ?>

```

Formulario que recuerda los valores enviados:

```

27 <form method="post" action="">
28     <label>Nombre:</label>
29     <input type="text" name="nombre"
30         value="<?php echo isset($_POST['nombre']) ? htmlspecialchars($_POST['nombre']) : ''; ?>"
31     >
32     <br><br>
33     <label>Aficiones:</label><br>
34     <input type="checkbox" name="aficiones[]" value="musica" <?php if (!empty($_POST['aficiones']) && in_array('musica', $_POST['aficiones']))
35         echo "checked"; ?>
36     > Música<br>
37
38     <input type="checkbox" name="aficiones[]" value="ninguna" <?php if (!empty($_POST['aficiones']) && in_array('ninguna', $_POST['aficiones']))
39         echo "checked"; ?>
40     > Ninguna<br>
41
42
43     <input type="checkbox" name="aficiones[]" value="deporte" <?php if (!empty($_POST['aficiones']) && in_array('deporte', $_POST['aficiones']))
44         echo "checked"; ?>
45     > Deporte<br>
46
47     <input type="checkbox" name="aficiones[]" value="lectura" <?php if (!empty($_POST['aficiones']) && in_array('lectura', $_POST['aficiones']))
48         echo "checked"; ?>
49     > Lectura<br><br>
50
51     <input type="submit" value="Enviar">
52 </form>

```

Salida:

Nombre:

Aficiones:

- ☐ Música
- ☐ Ninguna
- ☐ Deporte
- ☐ Lectura

Programa 29

Formulario más completo, con varias comprobaciones a la vez. Comprueba que el nombre no esté vacío, que se haya marcado al menos un módulo, y si no es así muestra el error en rojo. Además, mantiene los valores ya escritos para que el usuario no tenga que volver a teclear.

```

14  <?php
15
16  //Se muestrta solo si se ha pulsado el botón ENVIAR (isset)
17  // COMPROBACIÓN 1
18  // Compruebo que se haya seleccionado al menos un módulo y que el nombre NO esté vacío para MOSTRAR los datos,
19  // antes comprobabamos solo si se había pulsado enviar
20  if (!empty($_POST['modulos']) && !empty($_POST['nombre'])) {
21      $nombre = $_POST['nombre'];
22      $modulos = $_POST['modulos'];
23      print "Nombre: " . $nombre . "<br />";
24      foreach ($modulos as $modulo) { //Warning key undefined
25          print "Modulo: " . $modulo . "<br />";
26      }
27  } else {
28      ?> <!-- ACTION: le indicamos PHP_SELF ya que se va a REDIRIGIR a este mismo archivo -->
29      <form name="input" action="<?php $_SERVER['PHP_SELF']; ?>" method="post">
30          Nombre del alumno:
31          <!-- COMPROBACION 2: Si pulsas botón enviar con el nombre, te lo mantiene en el formulario-->
32          <input type="text" name="nombre" value="<?php if (isset($_POST['nombre']))
33              echo $_POST['nombre']; ?>" />
34
35          <?php
36          //COMPROBACION 3: En este caso, se ha pulsado el botón ENVIAR pero no se ha introducido un nombre:
37          if (isset($_POST['enviar']) && empty($_POST['nombre']))
38              //Imprimo el error en PHP
39              echo "<span style='color:red'> <-- Debe introducir un nombre!!</span>";
40          ?>
41
42          <p>Módulos que cursa:</p>
43
44          <?php // Muestro Error Si no están vacías tras pulsar el botón enviar
45          //COMPROBACION 4: Si pulsamos enviar sin escoger algún módulo (vacío): imprimo el ERROR
46          if (isset($_POST['enviar']) && empty($_POST['modulos']))
47              echo "<span style='color:red'> <-- Debe escoger al menos uno!!</span>"
48              ?>
49          <input type="checkbox" name="modulos[]" value="DWES" />
50          <?php
51          //COMPROBACIÓN 5: Si hemos seleccionado algún módulo, buscamos si este primer módulo fue seleccionado para marcarlo
52          if (isset($_POST['modulos']) && in_array("DWES", $_POST['modulos']))
53              echo 'checked="checked"';
54          ?>
55          Desarrollo web en entorno servidor<br />
56          <input type="checkbox" name="modulos[]" value="DWEC" />
57          <?php
58          //COMPROBACIÓN 6: Si hemos seleccionado algún módulo, buscamos si este primer módulo fue seleccionado para marcarlo
59
60          if (isset($_POST['modulos']) && in_array("DWEC", $_POST['modulos']))
61              echo 'checked="checked"';
62
63          ?>
64          Desarrollo web en entorno cliente<br />
65          <br />
66
67          <!-- Ahora el botón submit tiene un atributo name "boton_enviar" -->
68          <input type="submit" value="Enviar" name="boton_enviar" />
69      </form>
70  <?php
71  }
72  ?>
73 </body>

```

Salida:

Nombre del alumno:

Módulos que cursa:

- ☐ Desarrollo web en entorno servidor
- ☐ Desarrollo web en entorno cliente

Programa 30

Formulario de login con usuario y contraseña. Es solo educativo: nunca se debe almacenar ni mostrar una contraseña en claro. En la vida real se usa `password_hash` y `password_verify`, pero eso lo veremos más adelante.

```
1 <?php
2 // Programa.php - Usuario y contraseña (demo sin seguridad)
3 $usuario = $_POST['usuario'] ?? '';
4 $clave = $_POST['clave'] ?? '';
5 ?>
6 <!doctype html>
7 <html lang="es">
8 <head><meta charset="utf-8"><title>Programa29 - Login básico</title></head>
9 <body>
10 <h1>Programa - Formulario de acceso (demo)</h1>
11 <form method="post" action="">
12   <label>Usuario: <input type="text" name="usuario" value="<?php echo htmlspecialchars($usuario); ?>"></label><br>
13   <label>Contraseña: <input type="password" name="clave"></label><br>
14   <button type="submit">Entrar</button>
15 </form>
16 <?php if ($_SERVER['REQUEST_METHOD'] === 'POST'): ?>
17   <p>Has enviado usuario <strong><?php echo htmlspecialchars($usuario); ?></strong>.</p>
18   <small>Nota: Ejemplo educativo. No almacenar ni mostrar contraseñas en claro.</small>
19 <?php endif; ?>
20 </body>
```

Salida:

Programa - Formulario de acceso (demo)

Usuario:

Contraseña:

Programa 31

Formulario con radio buttons para elegir un turno (mañana, tarde o noche). Como son **radio**, solo se puede seleccionar uno. Después de enviar, el **checked** se mantiene en el que se haya elegido.

Programa31.php · líneas 1-19

```
1 <?php
2 // Programa.php - Radio buttons (turno)
3 $turno = $_POST['turno'] ?? '';
4 ?>
5 <!doctype html>
6 <html lang="es">
7 <head><meta charset="utf-8"><title>Programa30 - Radio</title></head>
8 <body>
9 <h1>Programa - Elegir turno (radio)</h1>
10 <form method="post" action="">
11 <label><input type="radio" name="turno" value="mañana" <?php if($turno==='mañana') echo 'checked'; ?>> Mañana</label><br>
12 <label><input type="radio" name="turno" value="tarde" <?php if($turno==='tarde') echo 'checked'; ?>> Tarde</label><br>
13 <label><input type="radio" name="turno" value="noche" <?php if($turno==='noche') echo 'checked'; ?>> Noche</label><br>
14 <button type="submit">Enviar</button>
15 </form>
16 <?php if ($turno !== ''): ?>
17 <p>Turno seleccionado: <strong><?php echo htmlspecialchars($turno); ?></strong></p>
18 <?php endif; ?>
19 </body>
```

Salida:

Programa - Elegir turno (radio)

- ☐ Mañana
 - ☐ Tarde
 - ☐ Noche
-

Programa 32

Formulario con checkboxes para elegir aficiones (música, deporte, lectura, cine, viajes). Aquí sí se pueden marcar varias, y PHP las recibe como un array porque el `name` es `aficiones[]`.

```
Programa32.php · líneas 1-23

1 <?php
2 // Programa.php - Checkboxes (aficiones)
3 $aficiones = $_POST['aficiones'] ?? [];
4 ?>
5 <!doctype html>
6 <html lang="es">
7 <head><meta charset="utf-8"><title>Programa31 - Checkboxes</title></head>
8 <body>
9 <h1>Programa - Aficiones (checkbox)</h1>
10 <form method="post" action="">
11 <?php
12     $lista = ['música','deporte','lectura','cine','viajes'];
13     foreach ($lista as $op) {
14         $checked = in_array($op, $aficiones) ? 'checked' : '';
15         echo "<label><input type='checkbox' name='aficiones[]' value='".htmlspecialchars($op)."' $checked> ".htmlspecialchars($op)."</label><br>";
16     }
17 ?>
18 <button type="submit">Enviar</button>
19 </form>
20 <?php if (!empty($aficiones)): ?>
21     <p>Seleccionadas: <strong><?php echo htmlspecialchars(implode(', ', $aficiones)); ?></strong></p>
22 <?php endif; ?>
23 </body>
```

Salida:

Programa - Aficiones (checkbox)

☐ música

☐ deporte

☐ lectura

☐ cine

☐ viajes

Enviar

Programa 33

Formulario con un **select** para elegir un país de una lista (España, Francia, Italia, Alemania, Portugal). Las opciones se imprimen con un **foreach** y se mantiene el **selected** en la opción que ya se haya elegido.

Programa33.php · líneas 1-25

```
1 <?php
2 // Programa.php - Select (país)
3 $pais = $_POST['pais'] ?? '';
4 $paises = ['España', 'Francia', 'Italia', 'Alemania', 'Portugal'];
5 ?>
6 <!doctype html>
7 <html lang="es">
8 <head><meta charset="utf-8"><title>Programa32 - Select</title></head>
9 <body>
10 <h1>Programa - Selección de país (select)</h1>
11 <form method="post" action="">
12   <label>País:
13     <select name="pais">
14       <option value="">-- Selecciona --</option>
15       <?php foreach ($paises as $p): ?>
16         <option value="<?php echo htmlspecialchars($p); ?>" <?php if($pais=== $p) echo 'selected'; ?>><?php echo htmlspecialchars($p); ?></option>
17       <?php endforeach; ?>
18     </select>
19   </label>
20   <button type="submit">Enviar</button>
21 </form>
22 <?php if ($pais != ''): ?>
23   <p>Has elegido: <strong><?php echo htmlspecialchars($pais); ?></strong></p>
24 <?php endif; ?>
25 </body>
```

Salida:

Programa - Selección de país (select)

País:

Programa 34

Validación de email con `filter_var($email, FILTER_VALIDATE_EMAIL)`.

Comprueba dos cosas: que el campo no esté vacío y que el formato sea correcto. Si falla, mete los errores en un array `$errores` y los pinta en rojo. Si todo está bien, muestra un mensaje de confirmación.

Lógica de validación:

Programa34.php · líneas 1-12

```
1 <?php
2 // Programa.php - Validación de email (formato)
3 $errores = [];
4 $email = $_POST['email'] ?? '';
5
6 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
7     if ($email === '') {
8         $errores[] = 'El email es obligatorio.';
9     } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
10         $errores[] = 'El formato de email no es válido.';
11     }
12 }
```

Formulario con los mensajes de error y confirmación:

Programa34.php · líneas 14-31

```
14 <!doctype html>
15 <html lang="es">
16 <head><meta charset="utf-8"><title>Programa36 - Validación email</title></head>
17 <body>
18 <h1>Programa - Validación de email</h1>
19 <?php if (!empty($errores)): ?>
20     <ul style="color:crimson;">
21         <?php foreach ($errores as $e): ?><li><?php echo htmlspecialchars($e); ?></li><?php endforeach; ?>
22     </ul>
23 <?php endif; ?>
24 <form method="post" action="">
25     <p><label>Email: <input type="email" name="email" value="<?php echo htmlspecialchars($email); ?>" placeholder="usuario@dominio.com"></label></p>
26     <button type="submit">Validar</button>
27 </form>
28 <?php if ($_SERVER['REQUEST_METHOD'] === 'POST' && empty($errores)): ?>
29     <p> Email válido: <strong><?php echo htmlspecialchars($email); ?></strong></p>
30 <?php endif; ?>
31 </body>
```

Salida:

Programa - Validación de email

Email:

Conclusiones de la entrega

Lo que me llevo de esta segunda parte:

- `match` me parece mucho más limpio que `switch`, lo voy a usar siempre que pueda.
- `foreach` es probablemente el bucle que más voy a usar.
- Separar las funciones en un `funciones.php` y reutilizarlas con `require_once` es una costumbre que conviene coger pronto.
- Los formularios siempre se validan, sin excepciones. No se puede confiar en lo que envía el usuario.
- `$_REQUEST` no se debe usar a la ligera; mejor `$_GET` o `$_POST` según el caso.
- Para autenticación, nunca contraseñas en claro.

La parte de formularios es la que más se va a usar en proyectos reales, así que es importante tenerla bien clara antes de seguir.